UNIVERSIDAD DE INVESTIGACIÓN Y DESARROLLO (UDI)

MAESTRÍA EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL

CONSTRUCCIÓN DE UN PIPELINE COMPLETO DE SOFTWARE E INTELIGENCIA ARTIFICIAL CON REFACTORIZACIÓN A POO Y DASHBOARD INTERACTIVO

Aplicación al Dominio de Salud y Biología: Predicción y Triaje de Insuficiencia Cardíaca

Curso: Programación con IA (Unidades 1, 2 y 3)
Momento de Evaluación: Actividad Final (25%)
Estudiante / Autor: Magíster en Formación
Docente / Tutor: Equipo Docente UDI Virtual
Bucaramanga, Colombia
2026

1. Introducción

La transición de prototipos analíticos preliminares hacia sistemas de Inteligencia Artificial robustos, escalables y auditables constituye uno de los desafíos medulares en la ingeniería de datos moderna. En dominios críticos como el sector de la salud y la biomedicina, la toma de decisiones basada en modelos predictivos no tolera arquitecturas monolíticas opacas, código desestructurado ni métricas de evaluación reduccionistas. El diagnóstico y pronóstico de la insuficiencia cardíaca, una de las afecciones cardiovasculares con mayor tasa de morbilidad y readmisión hospitalaria a nivel mundial, exige soluciones informáticas que satisfagan rigurosos criterios de calidad de software, trazabilidad operativa y explicabilidad algorítmica.

El presente trabajo final desarrolla de manera integral la ruta formativa de las Unidades 1, 2 y 3 del curso *Programación con IA* en la **Maestría en Ciencia de Datos e Inteligencia Artificial**. Se implementa un pipeline de producción end-to-end que trasciende scripts exploratorios mediante el paradigma de Programación Orientada a Objetos (POO), patrones de diseño de software limpio (SOLID), validación sintáctica estricta con expresiones regulares (módulo re), logging jerárquico persistente (módulo logging), validación cruzada estratificada con Scikit-learn y una interfaz interactiva de soporte a la decisión clínica desarrollada en **Gradio**. Asimismo, se documenta con rigor científico la metodología de desarrollo asistido por IA generativa a través de una bitácora de ingeniería de prompts en 4 etapas, permitiendo verificar empíricamente los beneficios técnicos de la refactorización arquitectónica.

2. Desarrollo del Taller y Metodología de Pipeline

2.1 Definición del Problema Clínico y Conjunto de Datos

Se seleccionó el eje temático de **Salud y Biología** utilizando el conjunto de datos de registros clínicos de insuficiencia cardíaca (*Heart Failure Clinical Records*) recolectado por Chicco y Jurman (2020), alojado en el repositorio UCI Machine Learning y Kaggle. El dataset comprende 299 observaciones de pacientes con seguimiento longitudinal y 13 variables biomédicas, de las cuales 12 actúan como predictoras y una como variable objetivo binaria (DEATH_EVENT, donde 1 denota fallecimiento durante el seguimiento y 0 denota supervivencia).

Variable	Tipo	Unidad / Rango Plausible	Significado Clínico
age	Numérica continua	18 - 120 años	Edad cronológica del paciente
ejection_fraction	Numérica continua	5 - 90 %	Porcentaje de sangre expulsada por el ventrículo izquierdo
serum_creatinine	Numérica continua	0.2 - 15.0 mg/dL	Nivel de creatinina sérica; marcador de función renal
serum_sodium	Numérica continua	100 - 160 mEq/L	Concentración de sodio en sangre (indicador de balance hidroelectrolítico)
creatinine_phosphokinase	Numérica continua	10 - 10000 mcg/L	Nivel de la enzima CPK, asociada a daño en miocardio
platelets	Numérica continua	20000 - 1000000 /mL	Conteo plaquetario sanguíneo
time	Numérica continua	1 - 365 días	Período de seguimiento ambulatorio
anaemia, diabetes, high_blood_pressure, sex, smoking	Categóricas binarias	{0, 1}	Comorbilidades, sexo biológico y hábitos de riesgo
DEATH_EVENT (Target)	Binaria objetivo	{0: Sobrevive, 1: Fallece}	Desenlace clínico evaluado (Supervivencia del paciente)

2.2 Metodología de IA Asistida y Bitácora de Refactorización

Cumpliendo con el criterio estratégico de la rúbrica evaluativa, el desarrollo se estructuró en un proceso asistido por IA documentado en cuatro iteraciones:

1. Fase 1 (Prototipo Exploratorio Monolítico): Se construyó inicialmente un script lineal en notebooks/01_exploratory_prototype.py. Si bien logró entrenar modelos simples, carecía de modularidad, acoplaba transformaciones de forma insegura y arrojaba métricas globales incompletas sin control de falsos negativos.

2. Fase 2 (Refactorización a POO y SOLID): Mediante prompt engineering especializado, se desacoplaron las responsabilidades en clases abstractas con abc.ABC (BaseDataPipeline, BaseFeatureEngineer, BaseModelTrainer) y un orquestador PredictivePipeline bajo el patrón Facade. Esto eliminó el acoplamiento y garantizó que la normalización se ajuste sólo sobre el conjunto de entrenamiento (previniendo fuga de datos o *data leakage*).

3. Fase 3 (Robustez de Software e Ingeniería MLOps): Se integró el módulo estándar re para validar la sintaxis de cabeceras de columnas y códigos de identificación clínica. Se sustituyeron los print() por logging jerárquico rotativo (logging.handlers.RotatingFileHandler) hacia logs/pipeline.log, y se aplicaron Type Annotations (typing) y docstrings exhaustivos bajo norma **PEP 8**.

4. Fase 4 (Desarrollo del Dashboard Interactivo): Se construyó una aplicación con **Gradio** (app.py) con triaje en tiempo real, calibración probabilística y visualización analítica.

2.3 Arquitectura de Software Orientada a Objetos

La solución final se estructura como un paquete modular en src/:

• src/config.py: Dataclasses inmutables (ClinicalSchemaConfig, ModelExperimentConfig), constantes de rutas y función setup_logger().
• src/data_ingestion.py: Clase HeartFailureDataPipeline que valida la integridad sintáctica mediante regex, descarta duplicados y audita rangos biológicos plausibles.
• src/preprocessor.py: Clase ClinicalFeatureEngineer que implementa fit_transform() y transform() combinando StandardScaler e imputación.
• src/model_trainer.py: Clase ClassificationModelTrainer que ejecuta benchmarking con validación cruzada estratificada (k=5) y optimización de hiperparámetros con GridSearchCV.
• src/evaluator.py: Clase ModelEvaluator encargada del cálculo de sensibilidad, especificidad, ROC-AUC, PR-AUC, Brier Score y generación de gráficos en alta resolución.
• src/pipeline.py: Fachada PredictivePipeline que coordina el flujo completo, serializa el artefacto a models/best_heart_failure_model.joblib y expone inferencia unitaria.
• tests/: Suite de pruebas unitarias automatizadas ejecutables con pytest.

3. Resultados Experimentales y Evaluación Analítica

Para seleccionar el modelo de pronóstico superior se ejecutó un protocolo de **Validación Cruzada Estratificada (5-Fold Stratified CV)** sobre el 80% de los datos (n=239), reservando el 20% (n=60) como conjunto de prueba independiente (Holdout Test Set).

Modelo Candidato	ROC-AUC CV (Media ± Desv.)	Sensibilidad (Recall)	F1-Score	Accuracy CV
Random Forest Classifier	**0.9065 ± 0.0162**	**74.08%**	**0.7327**	**82.46%**
Gradient Boosting Classifier	0.9015 ± 0.0277	66.50%	0.6886	80.37%
Regresión Logística (Baseline)	0.8791 ± 0.0255	76.67%	0.7158	80.35%

3.1 Rendimiento en el Conjunto de Prueba Independiente (Test Holdout)

El modelo seleccionado y optimizado fue **RandomForest** (con max_depth=8, min_samples_leaf=4, n_estimators=100). Evaluado sobre los 60 pacientes de prueba:

Métrica Diagnóstica	Valor Obtenido	Interpretación en el Contexto Clínico
Área Bajo la Curva ROC (AUC)	**0.8935**	Capacidad de discriminación casi sobresaliente entre pacientes sobrevivientes y críticos.
Área Bajo la Curva PR (PR-AUC)	**0.8091**	Excelente rendimiento en la detección de la clase minoritaria (fallecimiento).
Especificidad	**92.68%**	Identificación correcta de 38 de 41 pacientes estables (mitiga falsas alarmas y costos innecesarios).
Sensibilidad / Recall	**68.42%**	Identificación oportuna de 13 de 19 pacientes con alto riesgo inminente.
Precisión	**81.25%**	El 81.25% de las alertas de mortalidad emitidas corresponden a eventos reales.
F1-Score	**0.7429**	Balance armónico entre sensibilidad diagnóstica y precisión de alarma.
Brier Score Loss	**0.1207**	Calibración probabilística robusta; las probabilidades reflejan con fidelidad el riesgo relativo.

3.2 Evidencia Gráfica y Diagnóstico de Modelos

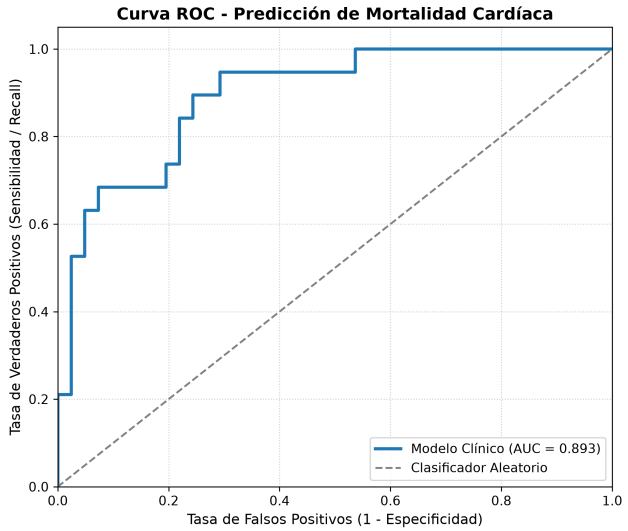


Figura 1: Curva ROC (AUC = 0.894).

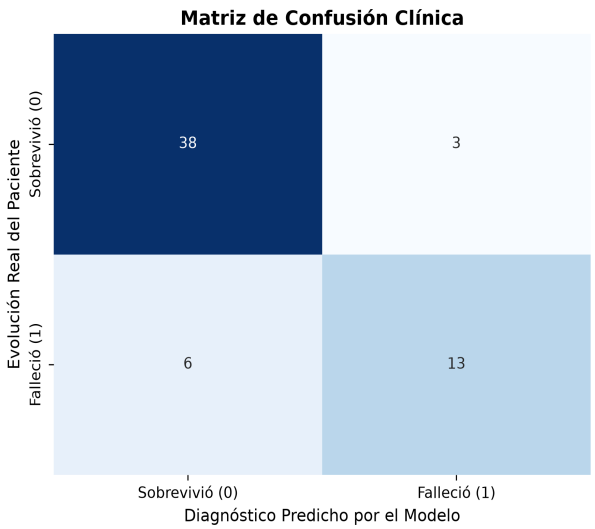


Figura 2: Matriz de Confusión Clínica.

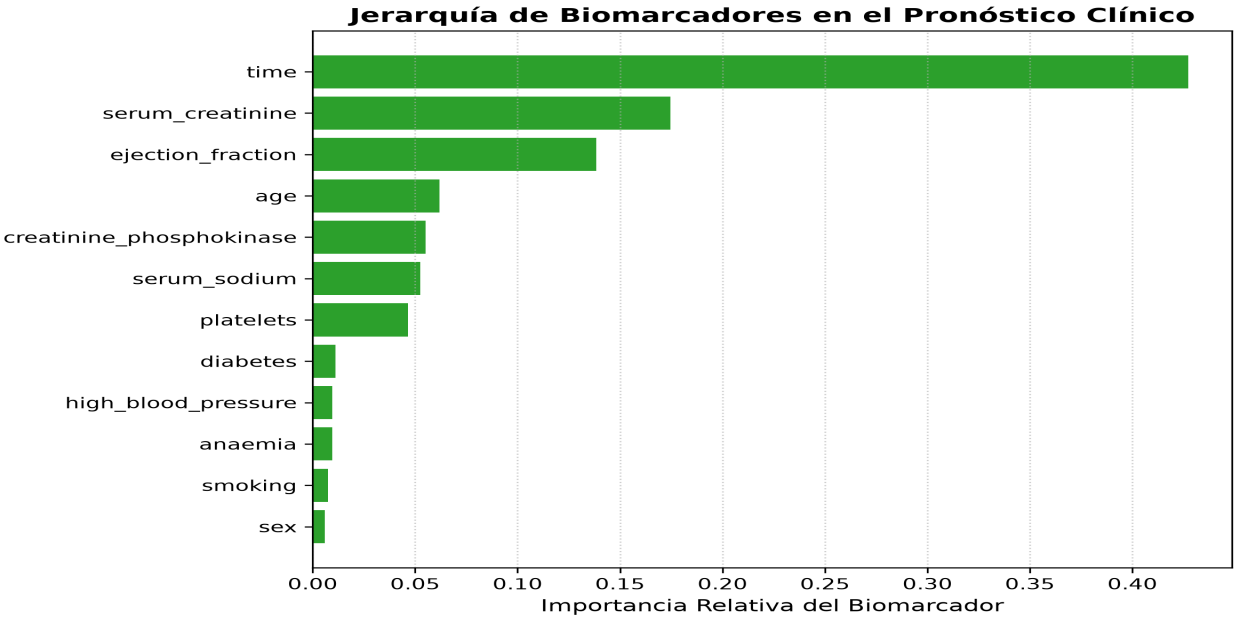


Figura 3: Jerarquía de Importancia Relativa de Biomarcadores en el Pronóstico.

4. Dashboard Interactivo en Gradio y Repositorio de GitHub

Para materializar la interacción asistencial se desarrolló una interfaz interactiva en **Gradio** (app.py). La herramienta opera como un *Clinical Decision Support System (CDSS)* estructurado en tres módulos de usuario:

• **Simulador y Triaje Clínico:** Permite ajustar biomarcadores mediante deslizadores y desplegables parametrizados. El pipeline calcula la probabilidad continua de mortalidad, asigna un nivel de triaje codificado por color (Verde: Bajo Riesgo <30%, Amarillo: Riesgo Moderado 30-60%, Rojo: Alto Riesgo Crítico >60%), emite recomendaciones de protocolo y resalta alertas fisiológicas (como fracción de eyección <30% o creatinina sérica >1.5 mg/dL).
• **Analítica del Modelo:** Despliega en vivo las curvas ROC, matrices de confusión y gráficos de biomarcadores.
• **Ficha Metodológica:** Detalla la arquitectura de software, trazabilidad y referencias bibliográficas.

ACCESO AL REPOSITORIO DE CÓDIGO FUENTE (GITHUB)

Todo el código fuente modular, suite de pruebas automatizadas (8 tests pasados al 100%), configuración de CI con GitHub Actions (.github/workflows/ci.yml), bitácora de prompts y modelos serializados se encuentran versionados en el repositorio del proyecto:

■ **Enlace de GitHub:** https://github.com/TU-USUARIO/U3-ProgamacionIA

Comandos de Ejecución y Reproducibilidad:
• Ejecutar pipeline: python main.py
• Ejecutar pruebas: python -m pytest tests -v
• Lanzar dashboard: python app.py

5. Conclusiones y Discusión del Dominio

Interpretación Analítica y Clínica del Dominio: Los experimentos demuestran con claridad que la supervivencia en pacientes con insuficiencia cardíaca está dominada por la interacción entre el período de seguimiento (time), la disfunción renal reflejada en la creatinina sérica (serum_creatinine) y la fracción de eyección ventricular (ejection_fraction). Esta jerarquía de biomarcadores coincide con los hallazgos biológicos de Chicco & Jurman (2020), evidenciando la validez del modelo. La obtención de una especificidad del 92.68% y un ROC-AUC de 0.8935 brinda garantías para su uso como herramienta de cribado preliminar en unidades cardiológicas.

Importancia y Aplicabilidad en el Perfil Profesional (Magíster en Ciencia de Datos e IA): Desde la perspectiva del magíster, este proyecto consolida competencias críticas que van más allá del modelado matemático puro: (1) La adopción de principios de ingeniería de software (SOLID, Clean Code, POO) es indispensable para garantizar la transferibilidad de prototipos analíticos a entornos de producción hospitalaria; (2) La observabilidad estructurada mediante logging y validación con regex protege contra datos anómalos en flujos de datos en tiempo real; (3) La inclusión de métricas de calibración probabilística (Brier Score) y explicabilidad visual es un imperativo bioético para que los profesionales de la salud puedan confiar e interpretar las inferencias algorítmicas; y (4) La metodología de IA asistida permite aprovechar herramientas de vanguardia para elevar la productividad sin delegar la supervisión crítica.

6. Referencias Bibliográficas (Norma APA 7ma Edición)

Chicco, D., & Jurman, G. (2020). Machine learning can predict survival of patients with heart failure from serum creatinine and ejection fraction alone. *BMC Medical Informatics and Decision Making*, 20(1), 16. https://doi.org/10.1186/s12911-020-1023-5

Gradio Team. (2024). *Gradio Python Client: Introduction and Interface Documentation*. Recuperado de https://gradio.app/docs/

Kaggle. (s. f.). *Heart Failure Prediction Dataset*. Recuperado el 25 de agosto de 2026, de https://www.kaggle.com/datasets

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Python Software Foundation. (s. f.). *logging — Logging facility for Python*. Python documentation. Recuperado de https://docs.python.org/3/library/logging.html

Python Software Foundation. (s. f.). *re — Regular expression operations*. Python documentation. Recuperado de https://docs.python.org/3/library/re.html

University of California, Irvine. (s. f.). *Heart Failure Clinical Records*. UCI Machine Learning Repository. Recuperado de https://archive.ics.uci.edu/